Predicting RNA–Protein Binding Sites: Reproducing & Extending the iDeepS Framework

Integrating RNA Sequence and Secondary Structure via Deep Learning

Team Members

Yadav Ankit Shivsurat Vishesh Vishwakarma

Karan Keer Avinash Kumar

CHE629 — Artificial Intelligence in Systems Biology

RNA-Binding Proteins (RBPs)

- **RBPs** attach to RNA and regulate post-transcriptional processes
- Key biological roles:
  - Alternative splicing
  - RNA stability & degradation
  - Translation regulation
  - RNA localization
- Experimental detection (CLIP-seq) is **expensive & noisy**
- Computational prediction enables large-scale analysis

The Classification Task

Input: RNA segment (101 nt)

Output:

$\mathbf{1}$ = Protein binds here

$\mathbf{0}$ = No binding

Separate model per RBP target

Why Sequence Alone Is Not Enough

Hairpin Loop — Accessible

```
...AU UGCAUG CA...
   ..  (((((( ..
```

Motif **exposed** → protein **CAN bind**

Buried in Stem — Inaccessible

```
...AU UGCAUG CA...
   ((  (((((( ((
```

Motif **hidden** → protein **CANNOT bind**

Core Insight: The same motif behaves differently depending on its **structural context**. This is the key idea behind iDeepS.

Real CLIP-seq Datasets

Dataset	Protein	Protocol	Train	Val	Test
17_ICLIP_HNRNPC_hg19	HNRNPC	iCLIP	24,000	6,000	10,000
10_PARCLIP_ELAVL1A_hg19	ELAVL1A	PAR-CLIP	24,000	6,000	10,000
28_ICLIP_TIA1_hg19	TIA1	iCLIP	24,000	6,000	10,000

Example FASTA Record

```
>seq_0001 label=1
AUGCUAGCUAGCUAGCUAGCUAGCUAGCUA...GCUAG                    (101 nt)
```

- All sequences: **fixed length 101 nt**; separate independent test file per dataset

Synthetic Sanity-Check Dataset

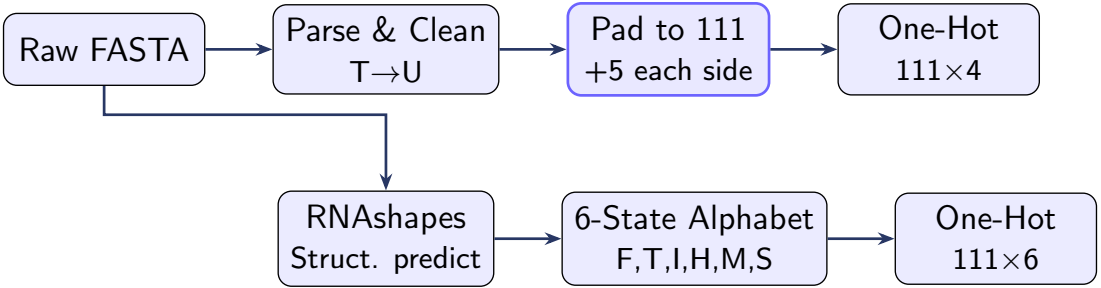
- **Purpose:** Controlled test where structure *must* matter
- Both classes contain the **same planted motif**
- **Positive:** motif in favorable structural context
- **Negative:** motif in unfavorable structural context
- Sequence-only model **should fail** ($\approx$75%)
- Structure-aware model **should succeed** ($\approx$100%)

Split	Samples
Train	8,000
Validation	2,000
Test	2,000

Validation Criterion

struct-aware $\sim$100%
vs seq-only $\sim$75%
$\Rightarrow$ structure **validated**

Data Processing Pipeline



6 Structure States (RNAshapes)

`F` Stem `T` Hairpin `I` Internal
`H` Hinge `M` Multiloop `S` Unpaired

Problem Formulation

Input: Seq (111×4) + Struct (111×6)
Output: $P(\text{bound}) \in [0,1]$
Loss: Binary cross-entropy

Overview of Evaluated Models

1. Baselines (No Structure)

- **Oli**: Linear SVM on 4-mer counts
- **DeepBind**: Sequence-only 1D-CNN

2. Main Architecture

- **iDeepS-like Model**: Parallel CNNs + BiLSTM predicting binding from Sequence + Structure

3. Novelty Extensions

- **Cross-Attention**: Sequences dynamically query structural context
- **MoE-SwiGLU**: Structure-routed mixture of SwiGLU experts for flexible decision rules

Benchmark 1: Oli (Linear SVM)

Architecture

- **Classical** ML baseline
- Features: 4-mer composition ($4^4 = 256$)
- **No deep learning, no structure**

Parameter Calculation

- Weight vector: 256 dimensions
- Bias: 1 value
- **Total learnable: 257 parameters**

Implementation (Python)

```
from sklearn.svm import LinearSVC

model = LinearSVC(
    class_weight="balanced",
    dual=False,
    random_state=629,
    max_iter=2000
)
model.fit(X_train, y_train)
test_prob = model.decision_function(X_test)
```

Benchmark 2: DeepBind (Seq CNN)

Architecture & Params (111×4 in)

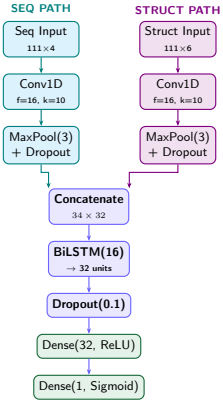
Layer	Params
Conv1D (f=16, k=10)	$4 \times 10 \times 16 + 16 = \mathbf{656}$
MaxPool + Dropout	0
Flatten ($34 \rightarrow 544$)	0
Dense(32, ReLU)	$544 \times 32 + 32 = \mathbf{17{,}440}$
Dense(1, sigmoid)	$32 \times 1 + 1 = \mathbf{33}$
Total	**18,129**

Keras Implementation

```
seq_input = layers.Input(shape=(111, 4))
x = layers.Conv1D(filters=16, kernel_size=10,
                  activation="relu")(seq_input)
x = layers.MaxPooling1D(pool_size=3)(x)
x = layers.Dropout(0.5)(x)
x = layers.Flatten()(x)
x = layers.Dense(32, activation="relu")(x)
output = layers.Dense(1, activation="sigmoid")(x)

model = models.Model(inputs=seq_input,
                     outputs=output)
```

iDeepS Architecture



Key Design Choices

- **CNN** (k=10): learns motifs up to 10 nt
- **MaxPool**(p=3): reduces $111 \rightarrow 34$ steps
- **Concatenate**: naive fusion of both paths
- **BiLSTM**(16): bidirectional context → 32 units
- **Dense**(32)→**Dense**(1): classifier head

vs DeepBind

iDeepS merges structural context *before* BiLSTM — allowing joint motif–structure reasoning.

iDeepS-like Model — Parameter Calculation

Layer-wise Parameter Count

Layer	Formula	Params
Conv1D-Seq (f=16, k=10)	$4\times10\times16+16$	656
Conv1D-Str (f=16, k=10)	$6\times10\times16+16$	976
MaxPool + Dropout	—	0
Concatenate $\rightarrow 34\times32$	—	0
BiLSTM(16)	$4\times(32+16)\times16\times2+4\times16\times2$	**6,272**
Dropout(0.1)	—	0
Dense(32, ReLU)	$32\times32+32$	1,056
Dense(1, Sigmoid)	$32\times1+1$	33
Total		**8,993**

BiLSTM Breakdown

Input dim $= 32$ (concat), LSTM units $= 16$

Per direction:

$$4 \times (32 + 16) \times 16 + 4 \times 16$$
$$= 4 \times 768 + 64 = 3{,}136$$

Wait — *bidirectional* $= 2$ directions:

Actually stored as 1 layer:

$$4 \times (32{+}16) \times 16 \times 2 + 4 \times 16 \times 2$$
$$= 6{,}144 + 128 = 6{,}272$$

+ Dense(32): $32 \times 32 + 32 = 1{,}056$
+ Dense(1): 33

Total: ∼9,000 params

Very lightweight — validates that gains come from **architecture**, not model size.

iDeepS Architecture — Code Snippet

Keras Implementation

```
# 1. Inputs
seq_input = layers.Input(shape=(111, 4), name="sequence_input")
struct_input = layers.Input(shape=(111, 6), name="structure_input")

# 2. Parallel CNN Feature Extraction
seq_branch = layers.Dropout(0.5)(layers.MaxPooling1D(pool_size=3)(
    layers.Conv1D(filters=16, kernel_size=10, activation="relu")(seq_input)))

struct_branch = layers.Dropout(0.5)(layers.MaxPooling1D(pool_size=3)(
    layers.Conv1D(filters=16, kernel_size=10, activation="relu")(struct_input)))

# 3. Concatenation and BiLSTM Sequence Modeling
merged = layers.Concatenate(axis=-1)([seq_branch, struct_branch])
bilstm_out = layers.Bidirectional(layers.LSTM(16, return_sequences=False))(merged)
bilstm_out = layers.Dropout(0.10)(bilstm_out)

# 4. Dense Classifier Output
dense_out = layers.Dense(32, activation="relu")(bilstm_out)
output = layers.Dense(1, activation="sigmoid")(dense_out)
```

Experimental Setup

Implementation Pipeline

1. Download 3 real + 1 synthetic dataset
2. Parse FASTA, extract labels, convert T→U
3. Split: 80% train / 20% validation
4. Encode sequence (111×4) + structure (111×6)
5. **Train 5 models $\times$ 4 datasets = 20 runs**
6. Save models, plots, predictions, metrics

Item	Count
Real datasets	3
Synthetic	1
Models	5
Experiments	20

Runtime

Kaggle GPU (T4)
with cached preprocessing
Optimizer: RMSprop
Loss: Binary Cross-Entropy

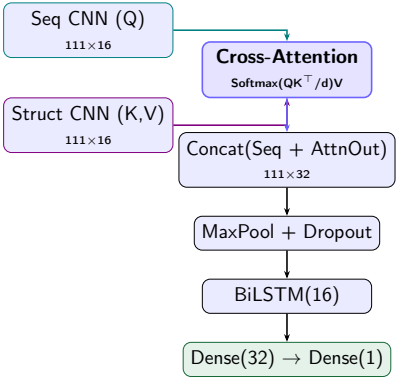
Novelty 1: Cross-Attention iDeepS

Motivation

- Standard iDeepS: **blind concatenation**
- No explicit interaction between seq & struct features
- Cross-attention: seq **queries** structure
- *Which structural context matters for this motif?*

Architecture Change

Conv1D(Seq)→**Q**
Conv1D(Struct)→**K, V**
Attention([Q,K,V])→Concat(Seq+Out)
MaxPool→BiLSTM→Dense(32)→Dense(1)



Cross-Attention iDeepS — Parameter Calculation

Layer-wise Parameter Count

Layer	Formula	Params
Conv1D-Seq (f=16,k=10,pad=same)	$4\times10\times16+16$	656
Conv1D-Str (f=16,k=10,pad=same)	$6\times10\times16+16$	976
Attention (Q,K proj)	$16\times16\times2$	512
Concat Seq+AttnOut ($\rightarrow$111×32)	—	0
MaxPool(3) + Dropout	—	0
BiLSTM(16)	~6,272*	6,272
Dropout(0.1)	—	0
Dense(32, ReLU)	$32\times32+32$	1,056
Dense(1, Sigmoid)	$32\times1+1$	33
Total		**~9,505**

*BiLSTM input=32, units=16, bidirectional

Attention Mechanism

$$\text{Attn}(Q,K,V)=\text{Softmax}\left(\frac{QK^\top}{\sqrt{d_k}}\right)V$$

$d_k=16$ (feature dimension)

Q projection: $16\times16=256$
K projection: $16\times16=256$
Attention params: 512

Overhead vs iDeepS

iDeepS $\approx 9{,}000$ params
Cross-Attn $\approx 9{,}500$ params
Overhead: ~6%
Justifies the gain in F1 & Recall

Cross-Attention iDeepS — Code Snippet

Keras Implementation

```
# 1. Base Convolutions (padding="same" aligns temporal dimension)
seq_conv = layers.Conv1D(filters=16, kernel_size=10, activation="relu",
                         padding="same")(seq_input)
struct_conv = layers.Conv1D(filters=16, kernel_size=10, activation="relu",
                            padding="same")(struct_input)

# 2. Cross-Attention: "Which structural context fits this motif?"
attention_out = layers.Attention(name="seq_to_struct_attention")([seq_conv, struct_conv])

# 3. Merge seq with attended structure, then Pool
merged_features = layers.Concatenate(axis=-1)([seq_conv, attention_out])
pooled_features = layers.MaxPooling1D(pool_size=3)(merged_features)
pooled_features = layers.Dropout(0.5)(pooled_features)

# 4. BiLSTM and Classification
lstm_out = layers.Bidirectional(layers.LSTM(16, return_sequences=False))(pooled_features)
lstm_out = layers.Dropout(0.10)(lstm_out)

x = layers.Dense(32, activation="relu")(lstm_out)
output = layers.Dense(1, activation="sigmoid")(x)
```

Novelty 2: Structure-Routed MoE-SwiGLU iDeepS

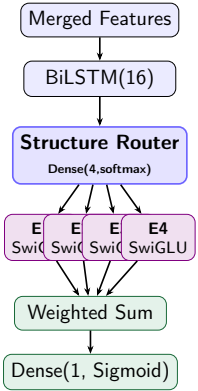
Motivation

- Single classifier head may be **too rigid**
- Different structures need **different decision rules**
- **Router** selects expert path via structure features
- **4 SwiGLU experts**: Dense(32)$\odot$Swish(Dense(32))

SwiGLU Formula

$\text{SwiGLU}(x) = \text{Dense}(x) \odot \text{Swish}(\text{Dense}(x))$
where $\text{Swish}(x) = x \cdot \sigma(x)$

Used in GPT-4, LLaMA, Gemini



MoE-SwiGLU iDeepS — Parameter Calculation

Layer-wise Parameter Count

Layer	Formula	Params
Conv1D-Seq (f=16,k=10,same)	$4\times10\times16+16$	656
Conv1D-Str (f=16,k=10,same)	$6\times10\times16+16$	976
Concat + MaxPool(3) + Drop	—	0
BiLSTM(16)	~6,272	6,272
Dropout(0.1)	—	0
Router Dense(4,softmax)	$32\times4+4$	132
Expert $\times4$: SwiGLU	$4\times(32\times32\times2+32\times2)$	8,448
Expert $\times4$: Dense(1)	$4\times(32+1)$	132
Dense(1, Sigmoid)	$32\times1+1$	33
Total		**~16,649**

SwiGLU Expert Detail

Each expert has 2 Dense(32) layers:

Gate: $32\times32+32=1{,}056$

Value: $32\times32+32=1{,}056$

Output Dense(1): $32+1=33$

Total per expert: $2{,}145$

$\times4$ experts $=8{,}580$ params

Model Size Comparison

Model	Params
Oli	257
DeepBind	18,129
iDeepS-like	~9,000
Cross-Attn	~9,500
MoE-SwiGLU	~16,649

Structure-Routed MoE-SwiGLU iDeepS — Code Snippet

1. Structure-Driven Router

```
# Route based on structural features
struct_pooled = layers.GlobalMaxPooling1D()(struct_conv)
router_weights = layers.Dense(4, activation="softmax")(struct_pooled)
router_reshaped = layers.Reshape((1, 4))(router_weights)
```

2. SwiGLU Experts

```
expert_outputs = []
for i in range(4):
    # SwiGLU: Dense(x) * Swish(Dense(x))
    gate = layers.Dense(32, activation="swish")(lstm_out)
    value = layers.Dense(32)(lstm_out)
    expert_out = layers.Multiply()([gate, value])

    # Reshape to stack along expert dimension
    expert_outputs.append(layers.Reshape((1, 32))(expert_out))

all_experts = layers.Concatenate(axis=1)(expert_outputs)
```

3. Weighted Combination

```
# Router shape:  (batch, 1, 4)
# Experts shape: (batch, 4, 32)
# Dot product yields weighted features
moe_out = layers.Dot(axes=(2, 1))(
    [router_reshaped, all_experts])

moe_out = layers.Flatten()(moe_out)

# Final Output
output = layers.Dense(1, activation="sigmoid")(moe_out)

model = models.Model(
    inputs=[seq_input, struct_input],
    outputs=output
)
```

Results — All Models on Real Datasets

Data	Model	Acc	Prec	Rec	F1	ROC	PR
HNRNPC	Oli (SVM)	.882	.655	.867	.746	.943	.766
	DeepBind	.936	**.840**	.842	.841	.975	.902
	iDeepS-like	.937	.822	.877	.849	**.977**	**.908**
	Cross-Attn	.932	.792	**.893**	.839	.976	.902
	MoE-SwiGLU	**.937**	.814	.889	**.849**	.976	.905
ELAVL1A	Oli (SVM)	.803	.504	**.802**	.619	.859	.614
	DeepBind	.893	**.775**	.659	.712	.895	.735
	iDeepS-like	.894	.776	.661	.714	.893	.735
	Cross-Attn	**.897**	.771	.692	**.729**	**.896**	**.744**
	MoE-SwiGLU	.894	.758	.688	.721	.893	.729
TIA1	Oli (SVM)	.807	.511	**.829**	.632	.888	.629
	DeepBind	.880	**.767**	.573	.656	.927	.753
	iDeepS-like	.882	.737	.634	.681	.927	.749
	Cross-Attn	.878	.751	.584	.657	.926	.758
	MoE-SwiGLU	**.887**	.758	.637	**.692**	**.932**	**.771**

Green = best per dataset–metric. **Highlighted** = novelty models.

Results — Synthetic Structure Validation

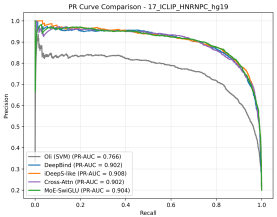
The definitive proof that structure encoding works

Model	Structure?	Accuracy	F1	ROC-AUC
Oli (SVM)	No	0.652	0.666	0.721
DeepBind (CNN)	No	0.749	0.798	0.768
iDeepS-like	Yes	1.000	1.000	1.000
Cross-Attention	Yes	1.000	1.000	1.000
MoE-SwiGLU	Yes	1.000	1.000	1.000

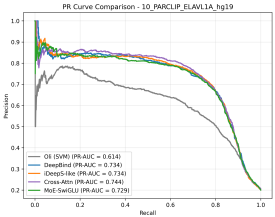
Conclusion

All structure-aware models achieve **perfect classification** on synthetic data.
Sequence-only models fail when binding is purely determined by structural context.

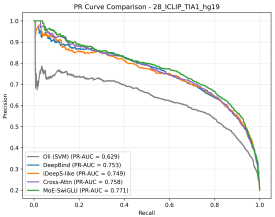
Precision-Recall (PR) Curve Comparisons — All Datasets



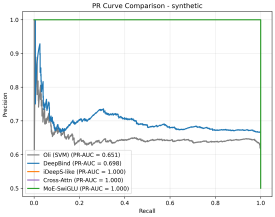
HNRNPC



ELAVL1A



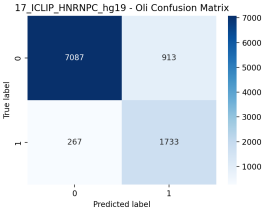
TIA1



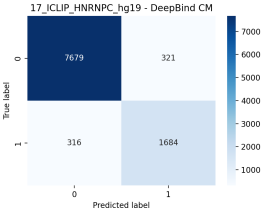
Synthetic

- PR-AUC better reflects performance on the minority positive class (4:1 imbalance)
- Structure-aware models consistently maximize precision and recall compared to baselines

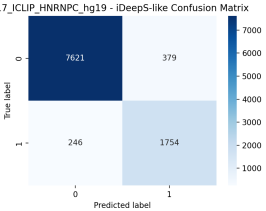
Confusion Matrices — HNRNPC Target



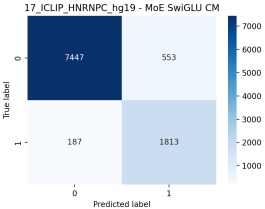
Oli (SVM)



DeepBind



iDeepS-like



MoE-SwiGLU

		Predicted Class	
		Negative (0)	**Positive (1)**
True	**Negative (0)**	**TN** (True Negative)	**FP** (False Positive)
	Positive (1)	**FN** (False Negative)	**TP** (True Positive)

- Progressive improvement in true-positive capture: Oli → MoE-SwiGLU
- Structure-aware models reduce false negatives significantly

Key Insights

What Worked

- Structure **genuinely helps**: 100% vs ∼75% on synthetic
- **MoE-SwiGLU** strongest overall (PR-AUC, F1 on TIA1 & HNRNPC)
- **Cross-Attn** improves recall and PR-AUC on HNRNPC

Important Caveats

- Accuracy Δ is small (∼1%)
- **F1, PR-AUC, Recall** reveal real gains
- Oli consistently weakest — confirms need for DL

Bottom line: Accuracy alone hides class-level improvements.
PR-AUC and Recall are the **true barometers** for biological classification.

Limitations & Future Work

Current Limitations

- **3 of 31** datasets (scoped reproduction)
- Structure depends on **RNAshapes accuracy**
- Fixed window (**101 nt**): misses long-range context
- No **cross-protein generalization**

Future Directions

- Scale to **all 31 datasets**
- Pre-trained **RNA language models** (RNA-FM)
- **Graph neural networks** for structure
- Multi-task learning across RBPs
- Attention **interpretability** for motifs

Conclusion

1. **Biology Validated**
   RNA–protein binding is a **sequence + structure** problem. Structure-blind models miss critical binding cues.

2. **Architecture Works**
   Dual-pathway CNN + BiLSTM consistently outperforms sequence-only baselines on real & synthetic data.

3. **Novelty Adds Value**
   Cross-attention and MoE-SwiGLU improve ranking metrics (PR-AUC, F1) beyond naive concatenation. MoE-SwiGLU is strongest overall.

4. **Metrics Matter**
   Accuracy hides class-level improvements. PR-AUC & Recall are true criteria for imbalanced biological classification.

Team Contributions

Sr.	Name	Roll Number	Contribution
1	Karan Keer	230531	Dataset preprocessing & Baseline models
2	Vishesh Vishwakarma	231166	Cross-Attention model design & training
3	Yadav Ankit Shivsurat	231181	MoE-SwiGLU model design & evaluation
4	Avinash Kumar	220256	iDeepS reproduction, results analysis & report

Submission Details

Report and code submitted by: Karan Keer

Course: CHE629 — Systems Biology & Deep Learning Session: 2025–26

Thank You